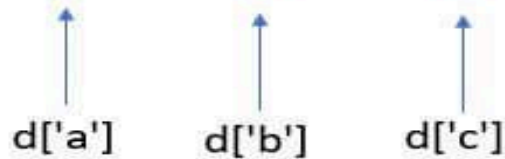


Dictionary in Python PYnative.com

Unordered collections of unique values stored in (Key-Value) pairs.

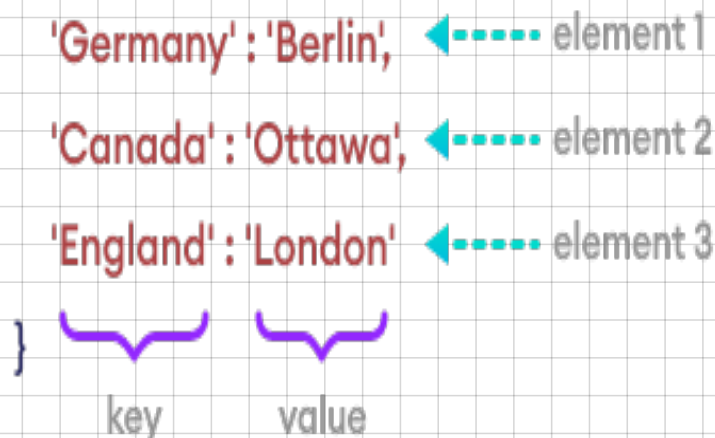
```
d = {'a': 10, 'b': 20, 'c': 30}
```



- ✓ **Unordered:** The items in dict are stored without any index value
- ✓ **Unique:** Keys in dictionaries should be Unique
- ✓ **Mutable:** We can add/Modify/Remove key-value after the creation

```
my_data = {  
    'Python dev': ['Sachin', 'Aman', 'Siya'],  
    'C++ dev': ['Rishu', 'Yashwant', 'Rahul'],  
    'PHP dev': ['Abhishek', 'Peter', 'Shalini'],  
    'detail': {  
        'name': ['Sachin', 'Siya'],  
        'hobby': ['Coding', 'Reading']  
    }  
}
```

```
country_capitals = {  
    'Germany': 'Berlin',  
    'Canada': 'Ottawa',  
    'England': 'London'  
}
```



DICTIONARY

["ginger", "lemon"]
0 1

["histark", "HOUNDHART"]
first - name

cheese_order = dict type = "Mozzarella"

cheese = "Large", size = "Large", sugar = 2

print(f"Cheese order: {cheese_order}")



type: Mozzarella, size: Large,
sugar: 2

cheese_recipe = {

cheese_recipe["base"] = "buck"

cheese_recipe["topping"] = "milk"

print(f"Recipe base: {cheese_recipe['base']}")
print(f"Recipe topping: {cheese_recipe['topping']}")

```
del choi = recipe["ingredient"]  
print(choi - recipe)
```

Block test

```
print('sugar' in choi - order)
```

FALSE

```
choi - order = {  
    "size": "Medium",  
    "type": "Ginger Cho",  
    "sugar": 1  
}
```

```
print(f"Order details | Keys: {choi - order.keys()}")
```

```
dict = Keys ["type", "size", "sugar"]
```

Values 1

Ginger Cho, Medium, 1

• items()

⇓
[('type', 'Ginger chai'), ('size',
'Medium'), ('sugar', 1)]

last_item = chai_order.pop_item()

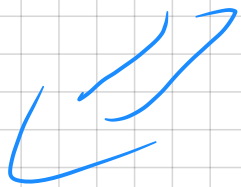
print(f"Remove last item:

{last_item}")

⇓
Removed last item: ("sugar",
1)

```
extra - species = { "wordworm": "crushed",  
"ginger": "sliced" }
```

```
char . recipe . update ( extra - species )  
print ( " ~ ~ ~ { char - recipe } "
```



```
"nose": block tea  
"wordworm": "crushed"  
"ginger": "sliced"
```

```
char . size = char . order [ "size" ]  
print ( f " ~ ~ ~ { char - size } "
```

```
char  
size is medium
```

```
char_size = char_order.get  
("note", "no note")
```

```
print(f"char size is  
{customer_note}")
```

no note

DICTIONAR (dict) în Python

Ce este un dicționar?

Colecție de perechi cheie-valoare (key-value)

Este mutabil

Cheile sunt unice

Se declară cu { }

Cum se declară

```
d = {  
  
    "nume": "Ana",  
  
    "varsta": 25,  
  
    "oras": "Bucuresti"  
  
}  
  
"nume" = cheie  
  
"Ana" = valoare
```

Accesare valori

```
print(d["nume"]) # Ana
```

Dacă cheia nu există eroare.

Variant sigur :

```
print(d.get("nume"))
```

Modificare / Adăugare

```
d["varsta"] = 26 # modificare
```

```
d[""] = " " # adăugare nouă cheie
```



MyScript Notes

tergere

```
d.pop("oras") # terge cheia
```

Sau:

```
del d["oras"]
```

Metode importante

```
keys()    print(d.keys())
```

```
values()  print(d.values())
```

```
items()   print(d.items())
```

Parcuregere

```
for key in d:
```

Sau

```
for k, v in d.items():
```

```
    print(key, d[key])
```

```
    print(k, v)
```

Exemplu complet

```
persoana = {  
    "nume": "Ion",  
    "varsta": 30  
}  
  
persoana["oras"] = "Cluj"  
  
for k, v in persoana.items():  
  
    print(k, v)
```

Caracteristici importante

Characteristic

Dict

Mutabil

Ordonat (Py 3.7+)

Chei unice

Acces prin index

(prin cheie)

SCHEM RAPID

DICT

{ key: value }

Chei unice

Mutabil

d[key]

get()

pop()

keys()

values()

items()

1. Care este diferența dintre d["key"] și d.get("key")?

Răspuns:

d["key"] d eroare dacă cheia nu există

d.get("key") returnează None (sau valoare default)

DIFERENȚA: LIST vs SET vs DICT

Ordonat

LIST

`l = [1, 2, 3]`

Mutabil

Permite duplicate

Acces prin index

SET

Neordonat

`s = {1, 2, 3}`

Fără duplicate

Mutabil

NU are index

DICT

Cheie-valoare

Chei unice

`d = {"a": 1, "b": 2}`

Mutabil

Acces prin cheie

DICTIONARY COMPREHENSION

Este varianta scurtă și Pythonică pentru a crea dicționare.

Sintaxă generală

`{c: expresie for c in iterabil}`

Exerci iul 1 – Accesare i modificare

```
d = {  
  
    "nume": "Ana",  
  
    "varsta": 25  
  
}
```

Afi eaz valoarea pentru "nume"

Modific "varsta" la 26

Adaug cheia "oras" cu valoarea "Cluj"

```
print(d["nume"])    # Ana
```

```
d["varsta"] = 26
```

```
d["oras"] = "Cluj"
```

```
print(d)
```

Rezultat final:

```
print(d["nume"])    # Ana
```

```
d["varsta"] = 26
```

```
d["oras"] = "Cluj"
```

```
print(d)
```

Exerci iul 2 – Parcurgere

```
d = {  
    "a": 1,  
    "b": 2,  
    "c": 3  
}
```

Afi eaz fiecare cheie i valoare.

Rezolvare

```
for k, v in d.items():
```

```
    print(k, v)
```

Output:

a 1

b 2

c 3

Exerci iul 3 – Verificare existen cheie

```
d = {"x": 10, "y": 20}
```

Verific dac "x" exist în dic ionar.

Rezolvare

```
print("x" in d) # True
```

in verific doar cheile, nu valorile.